

Our research in business goals has led us to adopt the categories shown in the list that follows. These categories can be used as an aid to brainstorming and elicitation. By employing the list of categories, and asking the stakeholders about possible business goals in each category, some assurance of coverage is gained.

1. Growth and continuity of the organization
2. Meeting financial objectives
3. Meeting personal objectives
4. Meeting responsibility to the employees
5. Meeting responsibility to society
6. Meeting responsibility to the state
7. Meeting responsibility to the shareholders
8. Managing market position
9. Improving business processes
10. Managing the quality and reputation of products
11. Managing change in the environment over time

19.4 Capturing ASRs in a Utility Tree

In a perfect world, the techniques described in Sections 19.2 and 19.3 would be applied early on in your development process: You would interview the key stakeholders, elicit their business goals and driving architectural requirements, and have them prioritize all of these inputs for you. Of course, the real world, lamentably, is less than perfect. It is often the case that you do not have access to these stakeholders when you need them, for organizational or business reasons. So what do you do?

Architects can use a construct called a *utility tree* when the “primary sources” of requirements are not available. A utility tree is a top-down representation of what you, as an architect, believe to be the QA-related ASRs that are critical to the success of the system.

A utility tree begins with the word “Utility” as the root node. Utility is an expression of the overall “goodness” of the system. You then elaborate on this root node by listing the major QAs that the system is required to exhibit. (You might recall that we said in Chapter 3 that QA names by themselves were not very useful. Never fear—they are only being used as intermediate placeholders for subsequent elaboration and refinement!)

Under each QA, record specific refinements of that QA. For example, performance might be decomposed into “data latency” and “transaction throughput” or, alternatively, “user wait time” and “time to refresh web page.” The refinements that you choose should be the ones that are relevant to your system. Under each refinement, you can then record the specific ASRs, expressed as QA scenarios.

Once the ASRs are recorded as scenarios and placed at the leaves of the tree, you can evaluate these scenarios against two criteria: the business value of the candidate scenario and the technical risk of achieving it. You can use any scale you like, but we find that a simple “H” (high), “M” (medium), and “L” (low) scoring system suffices for each criterion. For business

value, “high” designates a must-have requirement, “medium” identifies a requirement that is important but would not lead to project failure were it omitted, and “low” describes a nice requirement to meet but not something worth much effort. For technical risk, “high” means that meeting this ASR is keeping you awake at night, “medium” means meeting this ASR is concerning but does not carry a high risk, and “low” means that you have confidence in your ability to meet this ASR.

Table 19.1 shows a portion of an example utility tree. Each ASR is labeled with an indicator of its business value and its technical risk.

TABLE 19.1 Tabular Form of the Utility Tree for a System in the Healthcare Space

Quality Attribute	Attribute Refinement	ASR Scenario
Performance	Transaction response time	A user updates a patient’s account in response to a change-of-address notification while the system is under peak load, and the transaction completes in less than 0.75 seconds. (H, H)
	Throughput	At peak load, the system is able to complete 150 normalized transactions per second. (M, M)
Usability	Proficiency training	A new hire with two or more years’ experience in the business can learn, with 1 week of training, to execute any of the system’s core functions in less than 5 seconds. (M, L)
	Efficiency of operations	A hospital payment officer initiates a payment plan for a patient while interacting with that patient and completes the process with no input errors. (M, M)
Configurability	Data configurability	A hospital increases the fee for a particular service. The configuration team makes and tests the change in 1 working day; no source code needs to change. (H, L)
Maintainability	Routine changes	A maintainer encounters response-time deficiencies, fixes the bug, and distributes the bug fix with no more than 3 person-days of effort. (H, M)
		A reporting requirement requires a change to the report-generating metadata. Change is made and tested in 4 person-hours of effort (M, L)
	Upgrades to commercial components	The database vendor releases a new major version that is successfully tested and installed in less than 3 person-weeks. (H, M)
Security	Adding new feature	A feature that tracks blood bank donors is created and successfully integrated within 2 person-months. (M, M)
	Confidentiality	A physical therapist is allowed to see that part of a patient’s record dealing with orthopedic treatment, but not other parts or any financial information. (H, M)
	Resisting attacks	The system repels an unauthorized intrusion attempt and reports the attempt to authorities within 90 seconds. (H, M)
Availability	No down time	The database vendor releases new software, which is hot-swapped into place, with no downtime. (H, L)
		The system supports 24/7/365 web-based account access by patients. (M, M)

Once you have a utility tree filled out, you can use it to make important checks. For instance:

- A QA or QA refinement without any ASR scenario is not necessarily an error or omission that needs to be rectified, but rather an indication you should investigate whether there are unrecorded ASR scenarios in that area.
- ASR scenarios that receive a (H, H) rating are obviously the ones that deserve the most attention from you; these are the most significant of the significant requirements. A very large number of these scenarios might be a cause for concern regarding whether the system is, in fact, achievable.

19.5 Change Happens

Edward Berard said, “Walking on water and developing software from a specification are both easy if both are frozen.” Nothing in this chapter should be taken to assume that such a miraculous state of affairs is likely to exist. Requirements—whether captured or not—change all the time. Architects have to adapt and keep up, to ensure that their architectures are still the right ones that will bring success to the project. In Chapter 25, where we discuss architecture competence, we’ll advise that architects need to be great communicators, and this means great bidirectional communicators, taking in as well as supplying information. Always keep a channel open to the key stakeholders who determine the ASRs so you can keep up with changing requirements. The methods offered in this chapter can be applied repetitively to accommodate change.

Even better than keeping up with change is staying one step ahead of it. If you get wind of a change to the ASRs, you can take preliminary steps to design for it, as an exercise to understand the implications. If the change will be prohibitively expensive, sharing that information with the stakeholders will be a valuable contribution, and the earlier they know it, the better. Even more valuable might be suggestions about changes that would do (almost) as well in meeting the goals but without breaking the budget.

19.6 Summary

Architectures are driven by architecturally significant requirements. An ASR must have:

- A profound impact on the architecture. Including this requirement will likely result in a different architecture than if it were not included.
- A high business or mission value. If the architecture is going to satisfy this requirement—potentially at the expense of not satisfying others—it must be of high value to important stakeholders.

ASRs can be extracted from a requirements document, captured from stakeholders during a workshop (e.g., a QAW), captured from the architect in a utility tree, or derived from business goals. It is helpful to record them in one place so that the list can be reviewed, referenced, used to justify design decisions, and revisited over time or in the case of major system changes.

In gathering these requirements, you should be mindful of the organization's business goals. Business goals can be expressed in a common, structured form and represented as business goal scenarios. Such goals may be elicited and documented using PALM, a structured facilitation method.

A useful representation of QA requirements is a utility tree. Such a graphical depiction helps to capture these requirements in a structured form, starting from coarse, abstract notions of QAs and gradually refining them to the point where they are captured as scenarios. These scenarios are then prioritized, with this prioritized set defining your “marching orders” as an architect.

19.7 For Further Reading

The Open Group Architecture Framework, available at opengroup.org/togaf/, provides a complete template for documenting a business scenario that contains a wealth of useful information. Although we believe architects can make use of a lighter-weight means to capture a business goal, it's worth a look.

The definitive reference source for the Quality Attribute Workshop is [Barbacci 03].

The term *architecturally significant requirement* was created by the SARA group (Software Architecture Review and Assessment), as part of a document that can be retrieved at <http://pkcruchten.wordpress.com/architecture/SARAv1.pdf>.

The Software Engineering Body of Knowledge (SWEBOK), third edition, can be downloaded here: computer.org/education/bodies-of-knowledge/software-engineering/v3. As we go to press, a fourth edition is being developed.

A full description of PALM [Clements 10b] can be found here: https://resources.sei.cmu.edu/asset_files/TechnicalNote/2010_004_001_15179.pdf.

19.8 Discussion Questions

1. Interview representative stakeholders for a business system in use at your company or your university and capture at least three business goals for it. To do so, use PALM's seven-part business goal scenario outline, referenced in the “For Further Reading” section.
2. Based on the business goals you uncovered for question 1, propose a set of corresponding ASRs.

3. Create a utility tree for an ATM. (Interview some of your friends and colleagues if you would like to have them contribute QA considerations and scenarios.) Consider a minimum of four different QAs. Ensure that the scenarios that you create at the leaf nodes have explicit responses and response measures.
4. Find a software requirements specification that you consider to be of high quality. Using colored pens (real ones if the document is printed; virtual ones if the document is online), color red all the material that you find completely irrelevant to a software architecture for that system. Color yellow all of the material that you think *might* be relevant, but not without further discussion and elaboration. Color green all of the material that you are certain is architecturally significant. When you're done, every part of the document that's not white space should be red, yellow, or green. Approximately what percentage of each color did your document end up being? Do the results surprise you?

or customer representative) presents a system overview from a business perspective. This presentation should describe the following aspects of the project:

- The system's most important functions
- Any relevant technical, managerial, economic, or political constraints
- The business goals and context as they relate to the project
- The major stakeholders
- The architectural drivers (emphasizing architecturally significant requirements)

Step 3: Present the Architecture

The lead architect (or architecture team) makes a presentation describing the architecture at an appropriate level of detail. The “appropriate level” depends on several factors: how much of the architecture has been designed and documented, how much time is available, and the nature of the behavioral and quality requirements.

In this presentation, the architect covers technical constraints such as the operating system, platforms prescribed for use, and other systems with which this system must interact. Most importantly, the architect describes the architectural approaches (or patterns, or tactics, if the architect is fluent in that vocabulary) used to meet the requirements.

We expect architectural views, as introduced in Chapter 1 and described in detail in Chapter 22, to be the primary vehicle by which the architect conveys the architecture. Context diagrams, component-and-connector views, module decomposition or layered views, and the deployment view are useful in almost every evaluation, and the architect should be prepared to show them. Other views can be presented if they contain information relevant to the architecture at hand, especially information relevant to satisfying important quality attribute requirements.

Step 4: Identify the Architectural Approaches

The ATAM focuses on analyzing an architecture by understanding its architectural approaches. Architectural patterns and tactics are useful for (among other reasons) the known ways in which each one affects particular quality attributes. For example, a layered pattern tends to bring portability and maintainability to a system, possibly at the expense of performance. A publish-subscribe pattern is scalable in the number of producers and consumers of data, whereas the active redundancy pattern promotes high availability.

Step 5: Generate a Quality Attribute Utility Tree

The quality attribute goals are articulated in detail via a quality attribute utility tree, which we introduced in Section 19.4. Utility trees serve to make the requirements concrete by defining precisely the relevant quality attribute requirements that the architects were working to provide.

The important quality attribute goals for the architecture under consideration were named or implied in step 2, when the business goals were presented, but not with a degree of specificity that would permit analysis. Broad goals such as “modifiability” or “high throughput” or “ability to be ported to a number of platforms” establish context and direction, and provide a

backdrop against which subsequent information is presented. However, they are not specific enough to let us tell if the architecture suffices to achieve those aims. Modifiable in what way? Throughput that is how high? Ported to what platforms and in how much time? The answers to these kinds of questions are expressed as quality attribute scenarios representing architecturally significant requirements.

Recall that the utility tree is constructed by the architect and the project decision makers. Together, they determine the importance of each scenario: The architect rates the technical difficulty or risk of the scenario (on a H, M, L scale), and the project decision makers rate its business importance.

Step 6: Analyze the Architectural Approaches

The evaluation team examines the highest-ranked scenarios (as identified in the utility tree) one at a time; the architect is asked to explain how the architecture supports each one. Evaluation team members—especially the questioners—probe for the architectural approaches that the architect used to carry out the scenario. Along the way, the evaluation team documents the relevant architectural decisions and identifies and catalogs their risks, non-risks, and tradeoffs. For well-known approaches, the evaluation team asks how the architect overcame known weaknesses in the approach or how the architect gained assurance that the approach sufficed. The goal is for the evaluation team to be convinced that the instantiation of the approach is appropriate for meeting the attribute-specific requirements for which it is intended.

Scenario walkthrough leads to a discussion of possible risks and non-risks. For example:

- The frequency of heartbeats affects the time in which the system can detect a failed component. Some assignments will result in unacceptable values of this response; these are risks.
- The frequency of heartbeats determines the time for detection of a fault.
- Higher frequency leads to improved availability but also consumes more processing time and communication bandwidth (potentially leading to reduced performance). This is a tradeoff.

These issues, in turn, may catalyze a deeper analysis, depending on how the architect responds. For example, if the architect cannot characterize the number of clients and cannot say how load balancing will be achieved by allocating processes to hardware, there is little point in proceeding to any performance analysis. If such questions can be answered, the evaluation team can perform at least a rudimentary, or back-of-the-envelope, analysis to determine if these architectural decisions are problematic vis-à-vis the quality attribute requirements they are meant to address.

The analysis during step 6 is not meant to be comprehensive. The key is to elicit sufficient architectural information to establish some link between the architectural decisions that have been made and the quality attribute requirements that need to be satisfied.

Figure 21.1 shows a template for capturing the analysis of an architectural approach for a scenario. As shown in the figure, based on the results of this step, the evaluation team can identify and record a set of risks and non-risks, sensitivity points, and tradeoffs.

Scenario #: A12		Scenario: Detect and recover from HW failure of main switch.		
Attribute(s)	Availability			
Environment	Normal operations			
Stimulus	One of the CPUs fails			
Response	0.999999 availability of switch			
Architectural decisions	Sensitivity	Tradeoff	Risk	Nonrisk
Backup CPU(s)	S2		R8	
No backup data channel	S3	T3	R9	
Watchdog	S4			N12
Heartbeat	S5			N13
Failover routing	S6			N14
Reasoning	Ensures no common mode failure by using different hardware and operating system (see Risk 8) Worst-case rollover is accomplished in 4 seconds, as computing state takes that long at worst Guaranteed to detect failure within 2 seconds based on rates of heartbeat and watchdog Watchdog is simple and has proved reliable Availability requirement might be at risk due to lack of backup data channel . . . (see Risk 9)			
Architecture diagram	<pre>graph LR; In(()) --> P[Primary CPU OS1]; In --> B[Backup CPU with Watchdog OS2]; P --> S[Switch CPU OS1]; B --> S; P -.-> Heartbeat 1 sec B; S --> Out(())</pre>			

FIGURE 21.1 Example of architecture approach analysis (adapted from [Clements 01b])

At the end of step 6, the evaluation team should have a clear picture of the most important aspects of the entire architecture, the rationale for key design decisions, and a list of risks, non-risks, sensitivity points, and tradeoff points.

At this point, phase 1 is concluded.

Hiatus and Start of Phase 2

The evaluation team summarizes what it has learned and interacts informally with the architect during a hiatus of a week or so. More scenarios might be analyzed during this period, if desired, or answers to questions posed in phase 1 may be clarified.

Attendees at the phase 2 meeting include an expanded list of participants, with additional stakeholders joining the discussion. To use an analogy from programming: Phase 1 is akin to when you test your own program, using your own criteria. Phase 2 is when you give your program to an independent quality assurance group, who will likely subject your program to a wider variety of tests and environments.

In phase 2, step 1 is repeated so that the stakeholders understand the method and the roles they are to play. Then the evaluation leader recaps the results of steps 2–6, and shares the current list of risks, non-risks, sensitivity points, and tradeoffs. After bringing the stakeholders up to speed with the evaluation results so far, the remaining three steps can be carried out.

Step 7: Brainstorm and Prioritize Scenarios

The evaluation team asks the stakeholders to brainstorm quality attribute scenarios that are operationally meaningful with respect to the stakeholders' individual roles. A maintainer will likely propose a modifiability scenario, while a user will probably come up with a scenario that expresses ease of operation, and a quality assurance person will propose a scenario about testing the system or being able to replicate the state of the system leading up to a fault.

While utility tree generation (step 5) is used primarily to understand how the architect perceived and handled quality attribute architectural drivers, the purpose of scenario brainstorming is to take the pulse of the larger stakeholder community: to understand what system success means for them. Scenario brainstorming works well in larger groups, creating an atmosphere in which the ideas and thoughts of one person stimulate others' ideas.

Once the scenarios have been collected, they must be prioritized, for the same reasons that the scenarios in the utility tree needed to be prioritized: The evaluation team needs to know where to devote its limited analysis time. First, stakeholders are asked to merge scenarios they feel represent the same behavior or quality concern. Next, they vote for those scenarios they feel are most important. Each stakeholder is allocated a number of votes equal to 30 percent of the number of scenarios,¹ rounded up. Thus, if 40 scenarios were collected, each stakeholder would be given 12 votes. These votes can be allocated in any way that the stakeholder sees fit: all 12 votes for 1 scenario, 1 vote for each of 12 distinct scenarios, or anything in between.

The list of prioritized scenarios is compared with those from the utility tree exercise. If they agree, it indicates good alignment between what the architect had in mind and what the stakeholders actually wanted. If additional driving scenarios are discovered—and they usually are—this may itself be a risk, if the discrepancy is large. Such discoveries indicate some level of disagreement about the system's important goals between the stakeholders and the architect.

1. This is a common facilitated brainstorming technique.